

## 1. Goodbye 110, hello 210

The first recursive method you will write will perform some recursive `String` manipulation to upgrade every occurrence of "110" in a `String` to "210". It returns a `String` and takes in a single `String`. Some example usages below.

```
upgrade("lol110lol") --> "lol210lol"
upgrade("1110110") --> "1210210"
upgrade("10101010") --> "10101010"
upgrade("") --> ""
```

## 2. Twins once removed

The second task is to write a method that recursively counts identical `chars` in `String` that are separated by another `char`. We are looking for pairs of matching `chars` within a `String`. So, the `String` "kaki" would have one pair – the two k's. They are separated by the 'a' char. The `String` "aBaBa" has 3 total pairs. 2 pairs between 'a' chars and one pair of 'B's. Some example usages:

```
pairCount("kaki") --> 1
pairCount("10101010") --> 6
pairCount("zomg!1!1!") --> 3
pairCount("") --> 0
```

Pairs can also overlap, so we would expect the following:

```
pairCount("eeeeee") --> 3
```

## 3. To sum or not to sum

The final recursive method involves recursively determining if **any group** of elements in an array can be summed to some target, given some starting index. Some example usages:

```
sumToTarget(new int[]{8, 9, 5}, 0, 22) --> true
sumToTarget(new int[]{8, 9, 5}, 2, 22) --> false
sumToTarget(new int[]{8, 9, 5}, 0, 17) --> true
sumToTarget(new int[]{8, 9, 5}, 0, 18) --> false
sumToTarget(new int[]{8, 9, 5}, 0, 14) --> true
sumToTarget(new int[]{8, 9, 5}, 0, 13) --> true
sumToTarget(new int[]{8, 9, 5}, 1, 14) --> true
sumToTarget(new int[]{8, 9, 5}, 1, 13) --> false
```

`sumToTarget(new int[]{}, 0, 0) --> true`

### Hints and Notes:

- All of these problems could be solved without recursion, however, any solution that is not recursive will not receive credit.
- You will probably want to make use of some of the `String` methods we've been looking at in class like `charAt` and `substring`.